FoodBridge — React Frontend Mentorship

Day 6 Notes — ProtectedRoute, Role-Based Access, Bug Fixes

---

1. Why We Need ProtectedRoute

Without protection, anyone could type a dashboard URL directly (e.g. /donor/dashboard) and see it, even without logging in, or while logged in as the wrong role (e.g. an NGO viewing a Donor dashboard). ProtectedRoute is a wrapper component that checks login status and role before rendering the actual page.

2. Usage Pattern

```
<Route path="/donor/dashboard" element={
  <ProtectedRoute allowedRole="DONOR">
    <DonorDashboard />
  </ProtectedRoute>
} />
```

children = the actual protected page (e.g. DonorDashboard). allowedRole = a fixed string decided per route (NOT pulled dynamically from the logged-in user — that would defeat the purpose, since it would always match).

3. ProtectedRoute.jsx — Full Component

```
import { useContext } from "react"
import { Navigate } from "react-router-dom"
import { AuthContext } from "../context/AuthContext"

const ProtectedRoute = ({ children, allowedRole }) => {
  const { user } = useContext(AuthContext)

  if (!user) {
    return <Navigate to="/login" />
  }

  if (user.role !== allowedRole) {
    return <Navigate to="/login" />
  }

  return children
}

export default ProtectedRoute
```

Logic breakdown:

- if (!user) -> nobody is logged in at all -> redirect to /login immediately
- if (user.role !== allowedRole) -> someone IS logged in, but wrong role for this route -> also redirect
- return children -> only reached if both checks pass -> renders the actual protected page

4. Navigate vs useNavigate — When to Use Which

```
<Navigate to="..." />   -> a COMPONENT, returned directly during render.
                           Used when the redirect decision IS the render output
```

```
                          itself (e.g. ProtectedRoute's conditional checks).

useNavigate()           -> a HOOK returning a function, called as a SIDE EFFECT
                           (e.g. inside .then() after an API call succeeds).
                           Used in Login.jsx to redirect after login completes.
```

5. Mistakes Caught and Fixed

Mistake 1 — allowedRole set dynamically instead of fixed:

```
// WRONG — always passes, defeats the purpose
<ProtectedRoute allowedRole={user.role}>

// CORRECT — fixed per route
<ProtectedRoute allowedRole="DONOR">
```

Mistake 2 — wrong role assigned to NGO route:

```
// WRONG — NGO dashboard required DONOR role (backwards)
<Route path="/ngo/dashboard" element={
  <ProtectedRoute allowedRole="DONOR"><NgoDashboard /></ProtectedRoute>
} />

// CORRECT
<Route path="/ngo/dashboard" element={
  <ProtectedRoute allowedRole="NGO"><NgoDashboard /></ProtectedRoute>
} />
```

This bug would have blocked real NGOs from their own dashboard while letting Donors sneak in — caught during testing before it became a bigger problem.

Mistake 3 — missing .toLowerCase() in Login's redirect:

```
// WRONG — produced uppercase URL: /DONOR/dashboard
navigate(`/${decoded.role}/dashboard`)

// CORRECT — matches lowercase route paths defined in App.jsx
navigate(`/${decoded.role.toLowerCase()}/dashboard`)
```

Symptom: right after login, URL showed /DONOR/dashboard (broken/mismatched route). Navigating away and clicking the Navbar's Dashboard link 'fixed' it, because Navbar already correctly used .toLowerCase(). This confirmed the bug was specifically in Login.jsx's navigate call, not in routing or Navbar.

6. Verification Checklist Used

- Logged out, visited /ngo/dashboard directly -> redirected to /login (correct)
- Logged in as DONOR, visited /ngo/dashboard directly -> redirected to /login (correct, wrong role blocked)
- Logged in as DONOR, visited /donor/dashboard -> page rendered correctly
- Logged in fresh -> URL immediately showed lowercase /donor/dashboard, no extra navigation needed

7. Full Context API Recap (Requested Revision)

createContext() creates a channel/container object. Its built-in .Provider component is the only part actually renderable as JSX.

```
export const AuthContext = createContext()

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null)
  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  )
}
```

Activating app-wide (main.jsx):

```
<AuthProvider>
  <App />
</AuthProvider>
```

Reading context anywhere (must be called INSIDE a component function):

```
const { user, setUser } = useContext(AuthContext)
```

Where context is used across the app:

```
Login.jsx        -> setUser({token, role, email}) after successful login
AuthContext.jsx  -> useEffect restores user from localStorage on app startup
Navbar.jsx       -> reads user to show correct links; handleLogout calls setUser(null)
ProtectedRoute   -> reads user to verify login status and role before rendering a page
```

Recurring bug pattern to remember: setUser(...) does not update 'user' immediately (state updates are scheduled, not instant). Reading 'user' on the very next line still gives the OLD value. Fix: use the original source variable (e.g. decoded.role) instead of the state variable when data is needed right away.

8. Git Commit (End of Day 6)

```
git add .
git commit -m "feat: add ProtectedRoute for role-based access control

- Created ProtectedRoute.jsx in src/routes to guard dashboard pages
- Checks if user is logged in and if their role matches the route's allowedRole prop
- Redirects to /login via Navigate component if unauthenticated or wrong role
- Wrapped all four dashboard routes (Donor, NGO, Volunteer, Admin) with ProtectedRoute in App.jsx
- Fixed wrong allowedRole on NGO dashboard route (was DONOR, corrected to NGO)
- Fixed missing .toLowerCase() in Login redirect causing uppercase URL mismatch
- Verified protected routes correctly block unauthenticated and wrong-role access"
```

9. Next Steps (Day 7)

- Full revision session before adding new material
- Connect Register form to POST /users endpoint (build data object, handle organizationName -> null for non-NGO)
- Learn the ternary operator pattern: condition ? valueIfTrue : valueIfFalse
- Add success handling for Register (redirect to Login or show toast)
- Attach JWT automatically to protected requests via Axios interceptor
- Add auto-logout when token expires (using exp field from decoded JWT)